

Contents

Security Review with Fix Plan — StudioBook (fictional AI-built class-booking app)	1
Scope & method	1
Risk summary	1
Findings	1
B-01 — Critical	1
B-02 — Critical	2
B-03 — Critical	2
B-04 — Critical	3
B-05 — High	3
B-06 — High	4
B-07 — High	4
B-08 — Medium	5
B-09 — High	6
B-10 — Medium	6
B-11 — Medium	7
B-12 — Low	7
Remediation plan	8
Retest notes	8

Security Review with Fix Plan — StudioBook (fictional AI-built class-booking app)

Deliberately insecure for demonstration and training. Do not deploy. Run locally only.

Prepared: 2026-09-28 · **Tier:** Standard/Advanced · **Target:** targets/vibe-app (baseline)
vs. targets/vibe-app-fixed (retest target)

Scope & method

Same scope as RISK_SCAN_TOP10.md, extended to all 12 findings, with the differentiator this listing promises: an **AI fix prompt** per finding - the literal prompt used (via targets/vibe-app-fixed/AI_FIX_PROMPTS.md) to produce this repo's own fixed branch, not a hypothetical one written for the report. Every prompt below is copy-paste-ready for Claude Code, Cursor, or similar.

Risk summary

Severity	Count
Critical	4
High	4
Medium	3
Low	1
Total	12

Findings

B-01 — Critical

OWASP: A05:2021 - Security Misconfiguration · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: tool (gitleaks, semgrep)

Evidence: targets/vibe-app/.env:20; targets/vibe-app/components/AdminUserList.tsx:7

What's wrong. The Supabase service-role key (bypasses every RLS policy in the database) is read from a NEXT_PUBLIC_-prefixed environment variable inside a client component.

Why it matters. Next.js inlines every NEXT_PUBLIC_* variable into the JavaScript bundle shipped to every visitor's browser, logged in or not. Anyone who views page source gets a credential that can read or write ANY row in the database, unrestricted.

Exact fix. Move the query behind a server-only route using the service-role client correctly (never in a "use client" component); delete the NEXT_PUBLIC_ variable entirely.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
In targets/vibe-app-fixed/components/AdminUserList.tsx, remove the
Supabase client built directly in this "use client" component from a
NEXT_PUBLIC_-prefixed service-role var. Move the user-listing query
into a new server-only route that uses lib/supabase/admin.ts's
createAdminClient() (already server-only-guarded), and have this
component fetch from that route instead. Remove
NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY from .env/.env.example entirely
- only the non-prefixed SUPABASE_SERVICE_ROLE_KEY should exist.
After the change, `npm run build` then `grep -r "service_role"
.next/static` must find nothing.
```

B-02 — Critical

OWASP: A02:2021 - Cryptographic Failures · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: tool (gitleaks)

Evidence: targets/vibe-app/.env:3; targets/vibe-app/lib/stripe.ts:3

What's wrong. A Stripe secret key is hardcoded directly in a server file, and .env (holding the same value plus other credentials) is committed to the repository.

Why it matters. Anyone with read access to the repository - a contractor, a leaked backup, an over-shared CI log - obtains a live-shaped credential that can act as the application against Stripe's API.

Exact fix. Read the secret from an environment variable with no hardcoded fallback (fail to start if unset); add .env to .gitignore and stop committing it.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
In targets/vibe-app-fixed/lib/stripe.ts, remove the hardcoded
STRIPE_SECRET_KEY literal. Read it from
process.env.STRIPE_SECRET_KEY with no fallback string, and throw at
import time if it's unset, so the app fails to start rather than
silently running with no key. Add .env to
targets/vibe-app-fixed/.gitignore (remove the header comment there
that says it deliberately isn't listed) and delete the committed
targets/vibe-app-fixed/.env from the repo - .env.example is the only
committed template from here on. After the change, `git ls-files
targets/vibe-app-fixed | grep -x targets/vibe-app-fixed/.env` must
find nothing.
```

B-03 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization

Effort to fix: M

How found: tool (rls-checker)

Evidence: targets/vibe-app/supabase/migrations/20240101000002_rls_and_storage.sql:39

What's wrong. Row-level security is never enabled on the bookings table at all.

Why it matters. Any authenticated user can read, update, or delete EVERY other user's bookings directly through Supabase's own REST API, completely bypassing the Next.js app's own routes and any checks they perform.

Exact fix. Enable RLS with an owner/admin SELECT policy; explicitly revoke INSERT/UPDATE/DELETE from authenticated - every write must go through a service-role route or RPC, since a policy alone doesn't override Supabase's default table-level grants.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
Add a new migration in targets/vibe-app-fixed/supabase/migrations/
(don't edit the baseline migrations) that enables RLS on
public.bookings and adds a SELECT policy scoped to
`user_id = auth.uid() OR public.is_admin()`. Revoke INSERT, UPDATE,
DELETE on bookings from authenticated - all writes happen
server-side via the service-role client or an RPC. After the change,
scanners/rls-checker/rls_checker.py must report 0 FAIL for bookings.
```

B-04 — Critical

OWASP: A01:2021 - Broken Access Control · API3:2023 - Broken Object Property Level Authorization
Effort to fix: S

How found: tool (rls-checker)

Evidence: targets/vibe-app/supabase/migrations/20240101000002_rls_and_storage.sql:10

What's wrong. RLS IS enabled on profiles, but the SELECT policy is using (true) - equivalent to no policy at all for reads.

Why it matters. Every authenticated user can read every other user's profile row, including email and phone number - a full customer-PII leak.

Exact fix. Replace the policy with `id = auth.uid() OR is_admin()`; add a SECURITY DEFINER `is_admin()` helper to avoid the self-referential-policy recursion a plain subquery on profiles would hit.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
In the same new migration as B-03 (or a second one), replace
profiles_select_any_row_b04's `using (true)` with
`using (id = auth.uid() OR public.is_admin())`. Create
public.is_admin() as a SECURITY DEFINER, STABLE, search_path-pinned
function reading profiles.role for auth.uid(). This will make
app/studio/members/page.tsx and app/studio/dashboard/page.tsx's
member/booking queries silently return empty - fix both in this same
commit: members/page.tsx should select only {id, full_name} via the
service-role client (this is also the B-11 fix); dashboard queries
should read via the service-role client (the page is already
admin-gated). After the change, scanners/rls-checker/rls_checker.py
must report 0 FAIL for profiles, and both pages must still render
real data for an admin.
```

B-05 — High

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization
Effort to fix: S

How found: tool (semgrep)

Evidence: targets/vibe-app/app/api/bookings/[id]/cancel/route.ts:5

What's wrong. POST /api/bookings/{id}/cancel checks that the caller is logged in, but never checks that the booking being cancelled belongs to them.

Why it matters. Any authenticated user can cancel any OTHER user's booking by id - a denial-of-service against a specific victim's reservations.

Exact fix. Filter the update on id AND user_id (AND status, to make a second cancel a no-op); return the same 404 whether the booking belongs to someone else or doesn't exist.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
In app/api/bookings/[id]/cancel/route.ts, validate params.id is a
UUID (400 if not), then update via the service-role client with
.eq("id", id).eq("user_id", user.id).eq("status", "confirmed") -
return 404 if no row was affected, whether the booking belongs to
someone else or doesn't exist. After the change,
exploits/B-05-cancel-any-booking.sh (run against this fixed stack)
must show liam getting a 404 and maya's booking still confirmed.
```

B-06 — High

OWASP: A01:2021 - Broken Access Control · API5:2023 - Broken Function Level Authorization

Effort to fix: S

How found: manual

Evidence: targets/vibe-app/app/admin/page.tsx:6; targets/vibe-app/app/api/admin/grant-credits/route.ts:6

What's wrong. The admin-only 'grant credits' page hides its form from non-admins in the UI, but the route behind it never re-checks the caller's role server-side.

Why it matters. Any authenticated user can call the route directly and grant themselves - or anyone - unlimited free credits, bypassing the studio's entire paid-credit model.

Exact fix. Add a shared requireAdmin() helper (reads profiles.role via the service-role client, never client-supplied data) and call it before doing anything else in the route, returning 403 before even looking up the target user.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
Add a requireAdmin() helper (e.g. lib/auth/require-admin.ts) that
calls getSessionUser(), then looks up profiles.role for that user's
id via the service-role client (never trust user_metadata, request
body, or unverified JWT claims), returning the profile or null. In
app/api/admin/grant-credits/route.ts, call it and return 403 BEFORE
looking up the target user if the caller isn't an admin. Validate
amount is a bounded positive integer and targetUserId is a UUID; use
an atomic credit_balance = credit_balance + amount update, not
read-then-write. Reuse the same helper in /admin and
/studio/dashboard's page-level checks. After the change,
exploits/B-06-grant-credits-without-admin-role.sh must return 403
with liam's balance unchanged.
```

B-07 — High

OWASP: A08:2021 - Software and Data Integrity Failures · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: manual

Evidence: targets/vibe-app/app/api/stripe/webhook/route.ts:5

What's wrong. The Stripe webhook handler parses the request body directly with no signature verification at all.

Why it matters. Anyone who knows the webhook URL can POST an arbitrary forged event and have it processed as if Stripe sent it - including a fake `checkout.session.completed` that grants free credits (see B-08).

Exact fix. Verify the raw body against the stripe-signature header with `stripe.webhooks.constructEvent()` before parsing anything; fail closed if the webhook secret is unset.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
In app/api/stripe/webhook/route.ts, read the raw body with
`await request.text()` (not .json()), and call
stripe.webhooks.constructEvent(rawBody,
request.headers.get("stripe-signature"),
process.env.STRIPE_WEBHOOK_SECRET) inside a try/catch, returning 400
on any failure (missing header, wrong secret, tampered body) without
touching the database. Fail closed (throw at import time) if
STRIPE_WEBHOOK_SECRET is unset. After the change,
exploits/B-07-webhook-no-signature-check.sh must get HTTP 400 for
the unsigned forged event.
```

B-08 — Medium

OWASP: A04:2021 - Insecure Design · API8:2023 - Security Misconfiguration

Effort to fix: M

How found: manual

Evidence: targets/vibe-app/app/api/stripe/webhook/route.ts:11

What's wrong. The webhook handler has no dedup against the Stripe event's own id - POSTing the identical event body twice grants credits twice.

Why it matters. A replayed (or maliciously resubmitted) webhook event grants the same credits repeatedly - direct financial loss for the business, scaling with however many times the event is replayed.

Exact fix. Record processed event ids in a table with a unique constraint, and grant credits in the SAME atomic statement (a SECURITY DEFINER RPC) so there is no window where one succeeded without the other.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
Add a migration creating stripe_events(event_id text primary key,
checkout_session_id text unique not null, processed_at timestampz
default now()) (RLS enabled, zero policies, all grants revoked -
service_role only) and a SECURITY DEFINER function
apply_checkout_credits(p_event_id, p_session_id, p_user_id,
p_credits) that inserts into stripe_events with
`on conflict (event_id) do nothing returning event_id`; if no row
came back, return 'duplicate'; otherwise atomically update
profiles.credit_balance and return 'granted'. Revoke EXECUTE on this
function from public/anon/authenticated; grant only to service_role.
In the webhook route, require session.payment_status === "paid",
validate credits is a positive integer, and call this RPC instead of
the old select-then-update. After the change, a correctly-signed
event sent twice must move the balance by the credit amount exactly
once.
```

B-09 — High

OWASP: A03:2021 - Injection · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: manual

Evidence: targets/vibe-app/components/AvatarUpload.tsx:7; targets/vibe-app/supabase/migrations/20240101000002_rls_and_storage.sql:50

What's wrong. Avatar upload accepts any file type and size to a public storage bucket; an uploaded SVG with an embedded

is served back as image/svg+xml with no attachment disposition.

Why it matters. Stored XSS - a malicious SVG avatar executes its script in the browser of anyone who views it, including via the public bucket URL directly.

Exact fix. Restrict the bucket's allowed_mime_types to a real-image allow-list (no SVG) with a file size cap, and derive the stored filename's extension from the validated type, never the raw uploaded filename.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
Add a migration that UPDATES (not re-inserts - the bucket row
already exists) the avatars bucket to set file_size_limit (e.g. 2
MiB) and allowed_mime_types to {image/png,image/jpeg,image/webp} (no
image/svg+xml). In components/AvatarUpload.tsx, generate the storage
path server-side from the authenticated user's id and a fixed
extension derived from the validated MIME type (never from
file.name). After the change,
exploits/B-09-svg-avatar-stored-xss.sh must get a non-2xx response
for the SVG upload.
```

B-10 — Medium

OWASP: A04:2021 - Insecure Design · API4:2023 - Unrestricted Resource Consumption

Effort to fix: M

How found: manual

Evidence: targets/vibe-app/app/api/bookings/route.ts:20

What's wrong. The booking route accepts quantity with no server-side validation - a negative value is accepted and, combined with the duplicated pricing formula (B-12), actually INCREASES the caller's credit balance instead of charging them.

Why it matters. Any authenticated user can generate unlimited free credits for themselves with a single crafted request - direct financial loss.

Exact fix. Validate quantity is a positive integer within a sane bound before it reaches the database; move the booking + credit debit into one atomic, capacity/balance-checking database function.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
Add a migration with check (quantity > 0) on bookings and check
(credit_balance >= 0) on profiles, plus a SECURITY DEFINER function
book_class(p_user_id, p_class_id, p_quantity, p_cost) that: locks
the class row (select ... for update), checks remaining capacity
against confirmed bookings, checks credit_balance >= p_cost, then
atomically decrements the balance and inserts the booking, raising
an exception if either check fails. Revoke EXECUTE from
anon/authenticated, grant only to service_role. In
app/api/bookings/route.ts, validate quantity is a positive integer
within an upper bound before computing cost via the one shared
```

lib/pricing.ts function, then call this RPC. After the change, exploits/B-10-negative-quantity-booking.sh must return 400 with liam's balance unchanged.

B-11 — Medium

OWASP: A01:2021 - Broken Access Control · API3:2023 - Broken Object Property Level Authorization

Effort to fix: S

How found: manual

Evidence: targets/vibe-app/app/studio/members/page.tsx:11; targets/vibe-app/components/MemberNameList.tsx:4

What's wrong. The studio-members server component fetches (and ships to the browser in the React Server Component payload) every OTHER member's full profile row, including phone number - the page only ever renders their name.

Why it matters. Any member's browser receives every other member's email and phone number in the page's data payload, even though the rendered UI never shows it - trivially recoverable by inspecting network traffic.

Exact fix. Select only the columns the page actually renders ({id, full_name}) - fixing what data is FETCHED, not just what's displayed, closes the leak regardless of future rendering changes.

AI fix prompt (copy-paste into Claude Code / Cursor):

(Applied as part of the B-04 fix, in the same commit - see that entry. app/studio/members/page.tsx now selects only {id, full_name} instead of select("*"). After the change, a member's RSC flight payload must contain no other member's phone number or email.)

B-12 — Low

OWASP: N/A - maintainability, not a security control · N/A - maintainability, not a security control

Effort to fix: M

How found: manual

Evidence: targets/vibe-app/app/api/bookings/route.ts:24; targets/vibe-app/app/page.tsx:5; targets/vibe-app/app/studio/dashboard/page.tsx:4; targets/vibe-app/lib/pricing.ts:1

What's wrong. The credit-cost pricing formula is duplicated three times (client display, server charge, admin dashboard) with only one copy validating anything; Supabase queries are scattered inline across page components; the dashboard page is ~600 lines mixing data-fetching, aggregation and rendering; there are zero automated tests.

Why it matters. Not directly exploitable on its own, but this is exactly the surface where a fix for one bug (like B-10's quantity clamp) gets applied to one duplicated copy of the logic and silently missed in the others - as it literally did in the baseline.

Exact fix. One shared, VALIDATING pricing function; one data-access module for the dashboard; the page split into small components; tests covering the pricing/validation logic.

AI fix prompt (copy-paste into Claude Code / Cursor):

In lib/pricing.ts, change calculateCreditCost so the shared function VALIDATES (quantity must be a positive integer, throws otherwise) instead of clamping - clamping would silently charge for 1 seat on a negative quantity instead of rejecting it, reintroducing B-10's bug through the back door. Give the UI its own separate display-only clamp, importing the same validating function for the real calculation. Delete the two duplicate inline formulas in

app/api/bookings/route.ts and app/studio/dashboard/page.tsx and import from lib/pricing.ts instead. Split app/studio/dashboard/page.tsx into a data-loading module (lib/data/dashboard.ts) plus several smaller presentational components. Add vitest tests for lib/pricing.ts (valid/invalid quantity). After the change, `rg "credit\_cost\s\*\\*"` must match only inside lib/pricing.ts; `npm test` must pass; no single file under app/ should exceed ~150 lines.

Remediation plan

Sprint	Findings	Rationale
Sprint 1	B-01, B-02, B-03, B-04	Critical - active cross-tenant/cross-user data exposure or full compromise
Sprint 2	B-05, B-06, B-07, B-09	High - exploitable with realistic effort, real business impact
Sprint 3	B-08, B-10, B-11	Medium - narrower blast radius or requires a secondary condition
Backlog	B-12	Low - maintainability/hygiene, not directly exploitable

Retest notes

targets/vibe-app-fixed is this repo's own retest target - a separate Next.js app and Supabase project implementing every fix above (see its README's "What changed, per finding" table), built by literally applying the AI fix prompts above, one commit per finding id. Retest evidence: every script in targets/vibe-app-fixed/exploits/ reports FIXED with exit 0 against a freshly-reset stack; the M5 RLS checker reports 0 FAIL; M5's gitleaks/Semgrep scans are clean on the fixed app's tracked source (see scanners/results/ for the generated, current state of all of this - never trust this report's prose over those files).